



메뉴바를 클릭하면
계정을 관리할 수 있어요

카테고리 없음

[딥러닝 파이토치 교과서 교과서] #05 CNN(1)

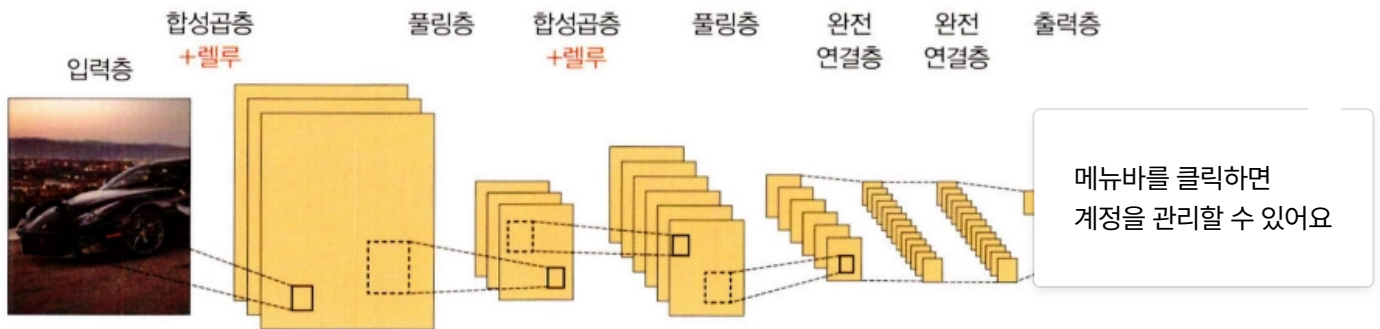
carchive-lab 2026. 9. 21. 19:42

1. 합성곱 신경망(Convolutional Neural Network, CNN 또는 ConvNet)

CNN은 이미지와 영상 같은 시각적 데이터를 분석하는 데 특화된 대표적인 딥러닝 알고리즘이다. 이미지 분석의 경우 데이터의 배열을 펼쳐서(flattening)이 처리하는 과정이 필요한데 이때 무시되는 공간적인 정보를 합성곱층이 보존할 수 있기 때문이다.

2. 구조

CNN은 입력층 → 합성곱층 → 풀링층 → 완전연결층 → 출력층으로 구성되어 있다. 각각 층이 어떠한 역할을 하는지 살펴보자.



2-1. **입력층** : (높이, 너비, 채널)의 2차원(흑백) 또는 3차원(컬러) 형태로 데이터를 받는다. 각 픽셀은 0부터 255사이의 값을 가지며, 높이와 너비채널의 경우 gray scale는 1의 값을, 컬러(RGB)이면 3의 값을 가진다. 채널은 때로는 깊이(depth)라고도 한다.

2-2. **합성곱층** : 이 층에서는 커널이나 필터를 사용하여 주요 특성 벡터(feature vector)를 추출한다. **커널 혹은 필터**는 이미지의 모든 영역을 훑으면서 특성을 추출하게 되는데 이렇게 추출된 결과물은 **특성맵**이다. 이때 다운샘플링이 발생한다. 커널 외에도 필터를 적용하는 위치와 간격인 **스트라이드**와 출력 데이터 크기를 조절을 위해 연산 전 0값을 넣는 **패딩** 등의 파라미터가 있다.

입력 데이터 : WHD, (W: 가로, H: 세로, D: 채널 또는 깊이)

하이퍼파라미터 : K(필터개수), F(필터크기), S(스트라이드), P(패딩)

출력 데이터 : $W2=(W1-F+2P)/S+1$, $H2=(H1-F+2P)/S+1$, $D2=K$

예시를 통해 연산 과정을 설명해보자면,

예시/1) WHD = (6, 6, 1), F= 3*3 , S=1 합성곱층



$$(1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$$

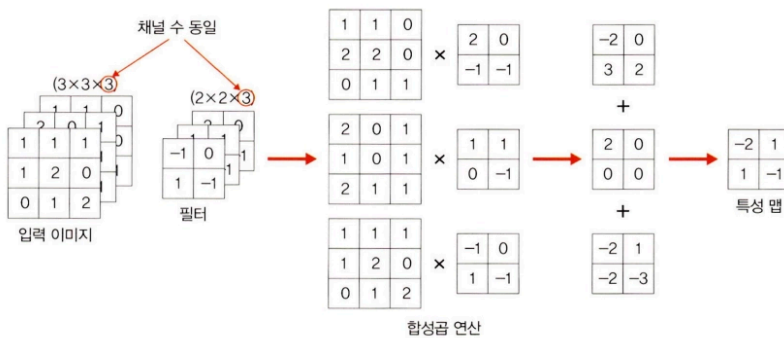




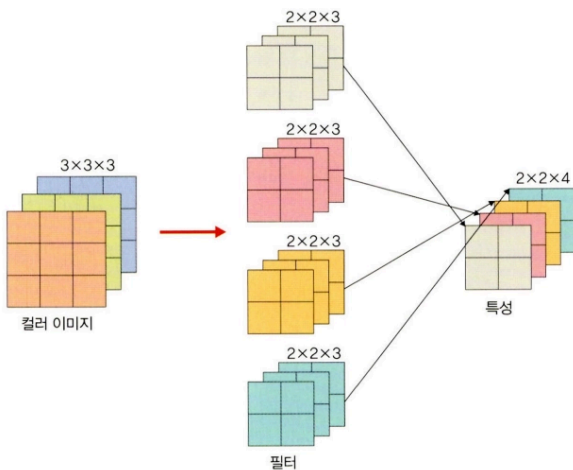
메뉴바를 클릭하면
계정을 관리할 수 있어요

연산 결과 (4, 4, 1) 크기의 특성 맵으로 줄었다.

예시/2) D=3 즉 컬러이미지의 합성곱. 이 경우에는 RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더한다. 이때 필터 채널이 3이라고 해서 필터 개수도 3개라고 오해하기 쉬운데, 실제로는 필터 개수가 하나이다.



예시/3) 필터가 두 개 이상이면, 필터 각각은 특성 추출의 결과의 채널이 된다.



2-3 풀링층(pooling layer) : 합성곱층과 유사하게 다운샘플링을 하지만 이와 달리 학습해야 할 가중치는 존재하지 않는다. 연산 방법으로는 **최대 풀링**, **평균 풀링**, **최소 풀링**이 있다. 각각 필터 내에서 최댓값 추출, 평균값 추출 그리고 최솟값을 추출하며, CNN은 합성곱층의 가중치를 보존하기 위해 최대 풀링을 사용한다.

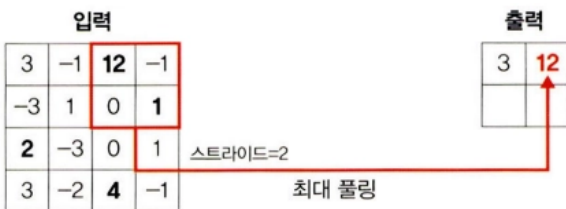
입력데이터 : $W1 \times H1 \times D1$

하이퍼 파라미터 : F, S

출력데이터 : $W2=(W1-F)/S+1, H2=(H1-F)S+1, D2=D1$

메뉴바를 클릭하면
계정을 관리할 수 있어요

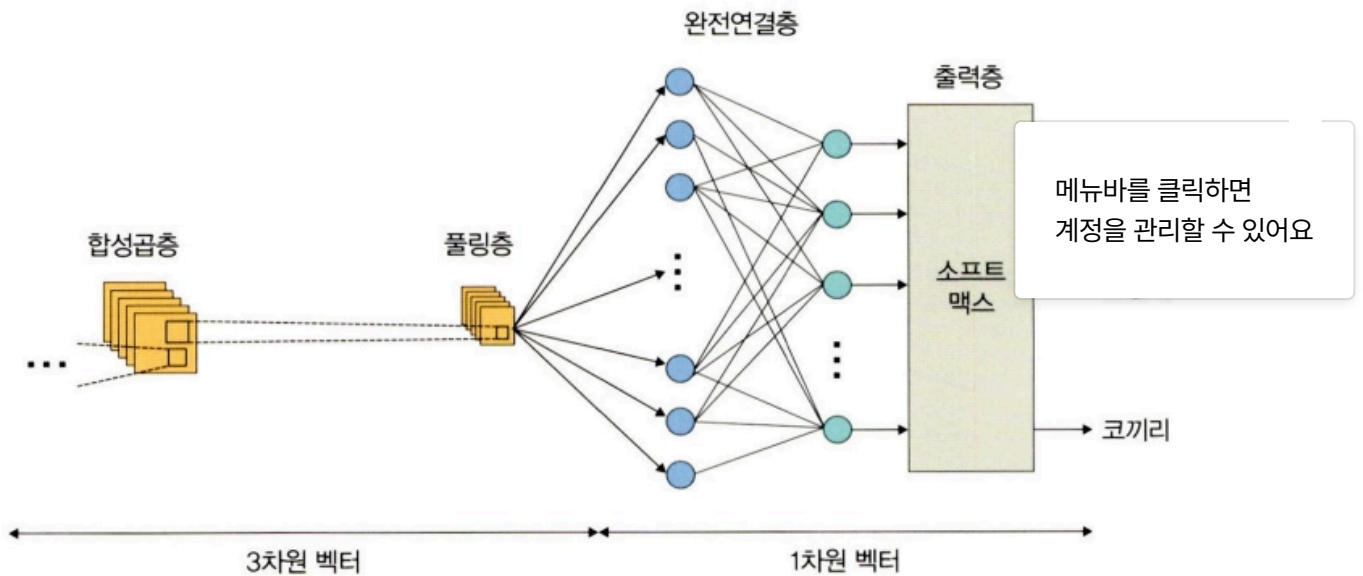
예시/1) 최대 풀링



같은 입력 데이터에 대하여 평균 풀링은 $[0, 3, 0, 1]$ 의 값을 가진다.

2-4. 완전 연결층(Fully Connected, FC) : 이 층에서 flattening이 발생한다.

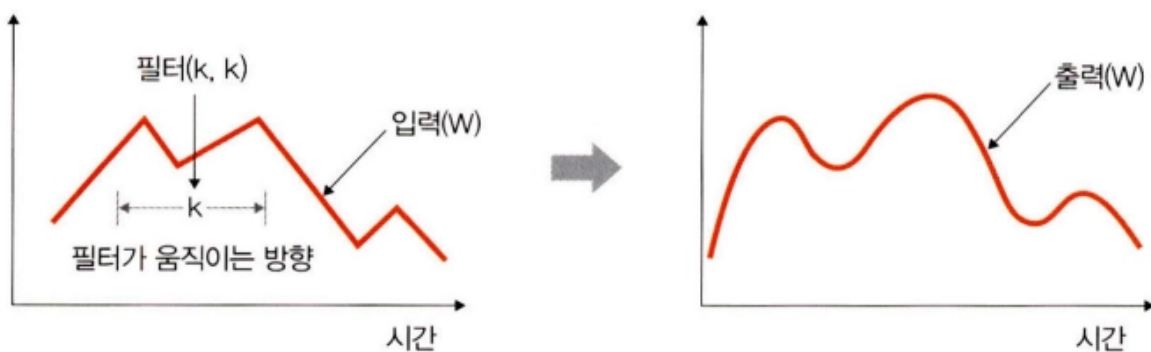
2-5. 출력층 : 소프트맥스 활성화함수를 사용해, 입력받은 값을 0~1 사이의 값으로 출력한다. 즉, 각 이미지가 어떤 레이블에 속할 확률을 값으로 출력하여 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정된다.



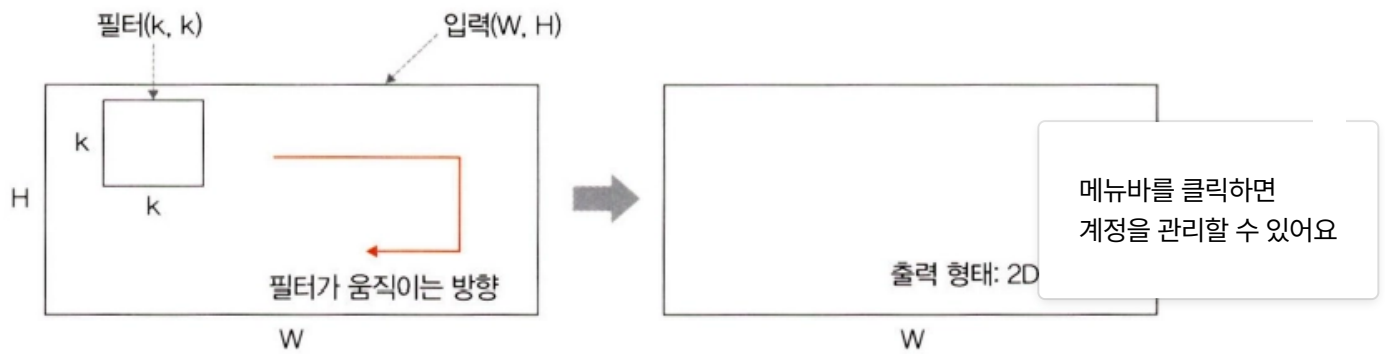
3. 종류

합성곱은 필터가 이동하는 방향과 출력 형태에 따라 1D, 2D, 3D 합성곱으로 나뉘며, 그외에도 3D 입력을 갖는 2D 합성곱이나 1x1 합성곱이 존재한다.

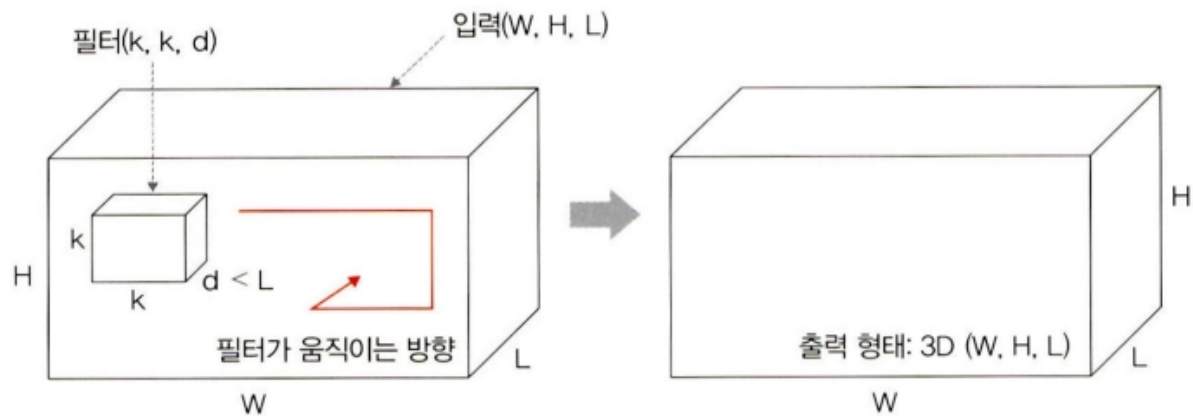
3-1. 1D 합성곱 : 가장 기본적인 형태로 필터는 시간을 축으로 좌우로만 움직인다. 예를 들어 입력이 [1, 1, 1, 1, 1]이고 필터가 [0.25, 0.5, 0.25]라면 [1, 1, 1]의 출력이 된다. 즉 출력 형태는 1차원 배열이며, 그래프 곡선을 완화할 때 사용된다. 음성 신호, 텍스트 문장, 시계열 주가 데이터에 사용된다.



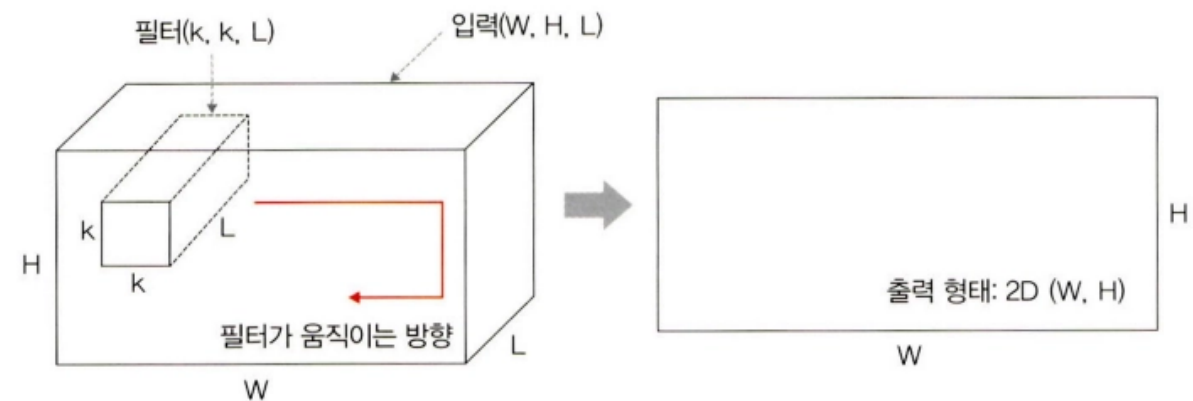
3-2. 2D 합성곱 : 두 방향(x, y)로 필터가 움직이며, 출력 형태는 2차원 행렬이다. 입력이 (W, H)이고 필터가 (k, k)의 형태라면 출력 역시 (W, H)이다. 흑백/컬러 이미지 등에 사용된다.



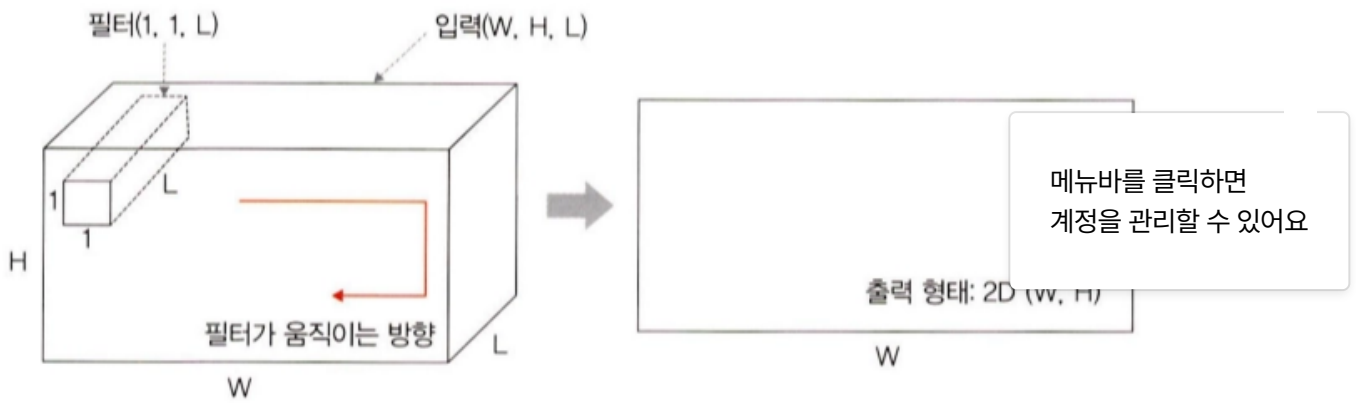
3-3. 3D 합성곱 : 필터는 (x, y, z) 의 세 방향으로 움직이며, 출력 형태는 3차원이다. 입력이 (W, H, L) 이고 필터가 (k, k, d) 인 경우 출력은 (W, H, M) 이다. 이때, d 가 L 보다 작아야 한다. 비디오 영상이나 CT/MRI 등의 3D 의료 영상에 사용된다.



3-4. 3D 입력을 갖는 2D 합성곱 : 입력이 3D일지라도 출력은 2D 행렬로 나오며, 그 이유는 L 이 입력 채널 수와 일치해야 하며, 실제 연산은 (x, y) 방향으로 수행되기 때문이다. 출력 (W, H, L) 에 필터(k, k, L)를 적용하면 출력은 (W, H) 이다. 예시로는 LeNet-5 나 VGG이다.



3-5. 1x1 합성곱 : 입력(W, H, L)에 필터($1, 1, L$)를 적용하며, 출력은 (W, H) 형태이다.



4. 실습

본 교재에서는 파이토치에서 제공하는 fashion_mnst 데이터셋을 예제로 사용하여 운동화, 셔츠, 샌들 등을 분류하는 실습을 진행하였다. 실습에 대한 코드는 깃허브에 업로드해두었으며, 본 글에서는 과정에 대해 간단히 설명하겠다.

```
0 : T-Shirt
1 : Trouser
2 : Pullover
3 : Dress
4 : Coat
5 : Sandal
6 : Shirt
7 : Sneaker
8 : Bag
9 : Ankle Boot
```

4-1. 라이브러리 호출

```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.autograd import Variable
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
```

4-2. fashion_mnst 데이터셋 내려 받기

```

train_dataset = torchvision.datasets.FashionMNIST("../chap05/data",
download=True,

transform=transforms.Compose([transforms.ToTensor()]))
test_dataset = torchvision.datasets.FashionMNIST("../chap05/data",
download=True,

train=False,
transform=transforms.Compose([transforms.ToTensor()]))

```

메뉴바를 클릭하면
계정을 관리할 수 있어요

4-3. 데이터를 데이터로더에 전달

```

train_loader=torch.utils.data.DataLoader(train_dataset, batch_size=100)
test_loader=torch.utils.data.DataLoader(test_dataset, batch_size=100)

```

4-4. 분류에 사용할 클래스 정의

```

labels_map = {0 : 'T-Shirt', 1: 'Trouser', 2 : 'Pullover', 3: 'Dress', 4
: 'Coat'
              , 5: 'Sandal', 6 : 'Shirt', 7 : 'Sneaker', 8 : 'Bag', 9 :
'Ankle Boot'}
fig = plt.figure (figsize=(8,8));
columns = 4;
rows = 5;
for i in range(1, columns*rows +1):
    img_xy = np.random.randint(len(train_dataset));
    img = train_dataset[img_xy][0][0,:,:]
    fig.add_subplot(rows, columns, i)
    plt.title(labels_map[train_dataset[img_xy][1]])
    plt.axis('off')
    plt.imshow(img, cmap='gray')
plt.show()

```

4-5. CNN 모델 생성

```

# CNN 모델 생성

class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential (

```

```

nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3,
padding=1),
# in_channels : 입력 채널의 수 / out_channels : 출력 채널의 수
# kernel_size : 일반적으로 정사각형으로 인식되며, 직사각형
하고 싶다면 (3, 5)처럼 지정 필요
# 스트라이드 명시가 없다면 기본적으로 (1, 1)
# padding : 패딩 크기로, 출력 크기를 조정하기 위해 입력 데이터 수
위에 0을 채운다.
nn.BatchNorm2d(32),
nn.ReLU(),
nn.MaxPool2d(kernel_size=2, stride=2)
# nn.MaxPool nn.AvgPool nn.MinPool 중에 최대 풀링 선택
)
self.layer2 = nn.Sequential(
nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
nn.BatchNorm2d(64),
nn.ReLU(),
nn.MaxPool2d(2)
)
self.fc1 = nn.Linear(in_features=64*6*6, out_features=600)
self.drop = nn.Dropout2d(0.25)
self.fc2 = nn.Linear(in_features=600, out_features=120)
self.fc3 = nn.Linear(in_features=120, out_features=10)

def forward(self, x):
out = self.layer1(x)
out = self.layer2(out)
out = out.view(out.size(0), -1) # out.size(0)는 100을 의미한다.
따라서 (100, ?) 크기의 텐서로 변경하겠다는 의미
out = self.fc1(out)
out = self.drop(out)
out = self.fc2(out)
out = self.fc3(out)
return out

```

메뉴바를 클릭하면
계정을 관리할 수 있어요

4-6. CNN 파라미터 정의

```

learning_rate = 0.001;
model = FashionCNN();

model.to(device)
criterion = nn.CrossEntropyLoss();

```

```
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);  
print(model)
```

4-6. CNN 학습

메뉴바를 클릭하면
계정을 관리할 수 있어요

```
num_epochs = 5  
count = 0  
  
loss_list = []  
iteration_list = []  
accuracy_list = []  
  
predictions_list = []  
labels_list = []  
  
for epoch in range(num_epochs):  
    for images, labels in train_loader:  
        images, labels = images.to(device), labels.to(device)  
  
        train = Variable(images.view(100, 1, 28, 28))  
        labels = Variable(labels)  
  
        outputs = model(train)  
        loss = criterion(outputs, labels)  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
        count += 1  
  
    if not (count % 50):  
        total = 0  
        correct = 0  
  
        for images, labels in test_loader:  
            images, labels = images.to(device), labels.to(device)  
            labels_list.append(labels)  
            test = Variable(images.view(100, 1, 28, 28))  
            outputs = model(test)  
            predictions = torch.max(outputs, 1) [1].to(device)  
            predictions_list.append(predictions)  
            correct += (predictions == labels).sum()  
            total += len(labels)
```

```
accuracy = correct * 100 / total
loss_list.append(loss.data)
iteration_list.append(count)
accuracy_list.append(accuracy)

if not (count % 500):
    print("Iteration:{}, Loss: {}, Accuracy:{}%".format(count,
loss.data, accuracy))
```

메뉴바를 클릭하면
계정을 관리할 수 있어요

공감

'카테고리 없음'의 다른글

이전글 [딥러닝 파이토치 교과서] #02. 파이토치 기초

현재글 : [딥러닝 파이토치 교과서 교과서] #05. CNN(1)

carchive-lab 님의 블로그

carchive-lab 님의 블로그 입니다.

댓글 0



carchive-lab

내용을 입력하세요.

등록



carchive-lab 님의 블로그
carchive-lab 님의 블로그 입니다.

글쓰기

블로그 관리

메뉴바를 클릭하면
계정을 관리할 수 있어요

분류 전체보기 (3)

Deep Learning (2)

Tag

머신러닝 #딥러닝

최근글 인기글

[딥러닝 파이토치 교과서 교과서] #05.
CNN(1)
2026.09.21 19:42

[딥러닝 파이토치 교과서] #02. 파이토치
기초
2026.09.07 23:15

[딥러닝 파이토치 교과서] #01. 머신러닝
과 딥러닝
2026.09.07 21:53

최근댓글

공지사항

메뉴바를 클릭하면
계정을 관리할 수 있어요

Archives

2026/09

〈 2026.09 〉						
일	월	화	수	목	금	토
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

Total

3

Today : 0

Yesterday : 0

검색내용을 입력하세요.